

創成デザイン工学及び演習

2026/06/16

Yuichi Nakamura / Chie Mizuta

Today's Topic

- API がどういうものか理解する
- API を2つかけ合わせて自分が作りたいサービスを考える
- API の使い方を理解する
- コードを書いて API にリクエストしてみる

Why did you decide to learn programming?

What got me into programming

撮った写真にコメントつけられるような Web サイト
作ってくんない？（頼まれただけ）



どうやって作ればいいのか...

What got me into programming

撮った写真にコメントつけられるような Web サイト
作ってくんない？（頼まれただけ）



どうやって作ればいいのか...

まずは調べる、わからなければ聞く、そして真似して書く

※ 調べ方がわからない場合は最初に聞くほうが良い

Way to Learn Programing

1. 挙動が似ているプログラムを検索して、まずは真似をして書いてみる
2. 真似した結果動かないなら、なぜ動かないかまた調べる
 - デバッグしながら書く
 - エラーログで検索をする
3. 動くまでくり返す(書いて消してをくり返す)
4. 動いてからソースコードを読み直して理解する
 - 関数やメソッドを更に調べる
 - 自分なりにコメントをたくさん書いておく

Way to Learn Programing

1. 挙動が似ているプログラムを検索して、まずは真似をして書いてみる
2. 真似した結果動かないなら、なぜ動かないかまた調べる
 - デバッグしながら書く
 - エラーログで検索をする
3. 動くまでくり返す(書いて消してをくり返す)
4. 動いてからソースコードを読み直して理解する
 - 関数やメソッドを更に調べる
 - 自分なりにコメントをたくさん書いておく

ひたすら書いては消すを
繰り返す



書いた量で覚える

Next pain point

ある程度大きなサービスの構築

- 要件定義
- サービスのデザイン
- サイト設計(サイト構造、画面設計)
- データベースの設計
- 言語何で作る …etc

Next pain point

ある程度大きなサービスの構築

- 要件定義
- サービスのデザイン
- サイト設計(サイト構造、画面設計)
- データベースの設計
- 言語何で作る ...etc

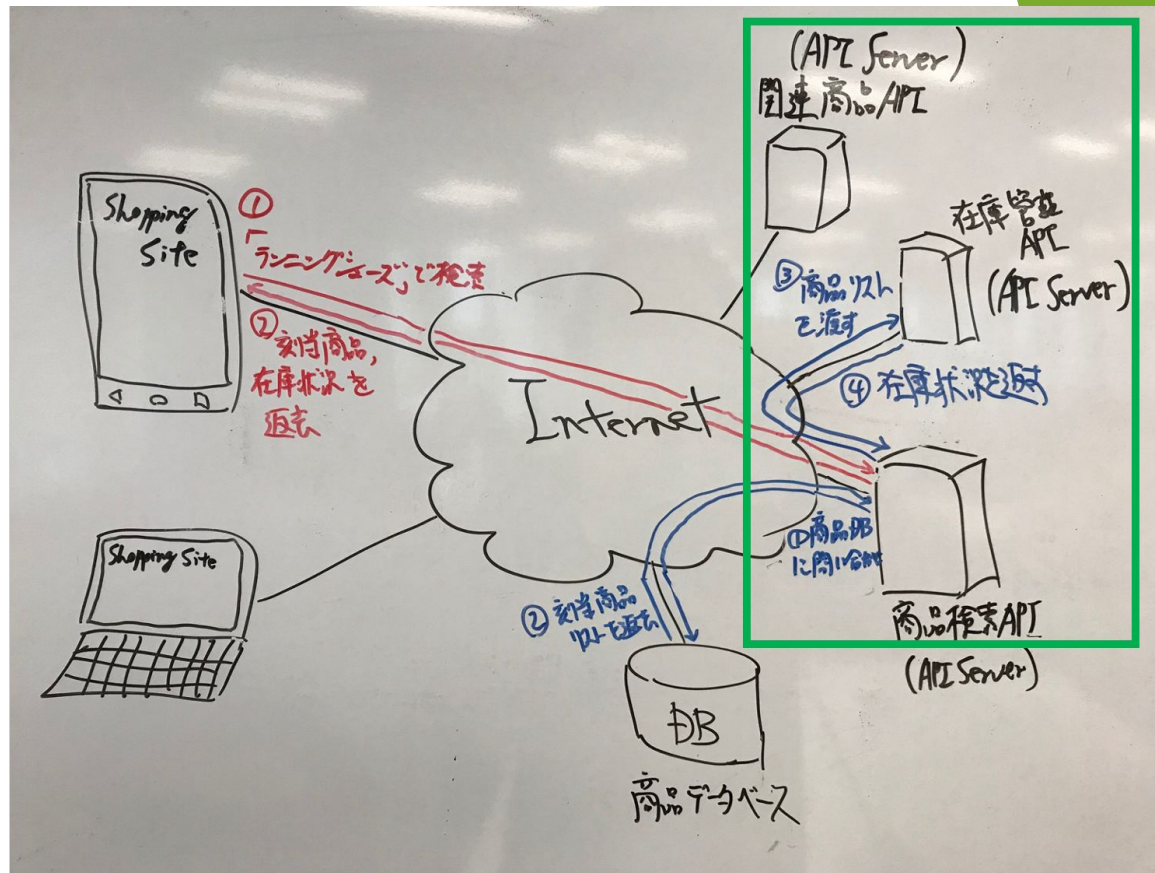
全部自分たちだけで作るのは相当大変 😱

→ 部分的に手伝ってるものって何かないのかな？ 🤔

What is API

- Application Programming Interface
- ソフトウェアの一部をインターネット上に公開して機能を共有すること
 - 主に公開するのは利用方法とレスポンス形式
 - 利用方法にはルールなども含まれる
 - プログラムの中身は公開しない

Example



Reason to use API

- メリット
 - 開発が効率化できる
 - 最新の情報が自動的に取得できる
 - 自分で管理しなくても良い
 - セキュリティ管理
 - サーバの管理 ...etc
- デメリット？ / Cons?
 - 急に使えなくなることがある

多種多様な API

- 天気予報
- ショッピング
- SNS
- 画像系
- 動画系
- 音声認識



- カレンダー
- 地図
- 形態素解析
- ヘルスケア
- 音楽系
- メモ



...etc



Let's start using APIs

How to proceed tasks

<https://github.com/sonatax/class-2025/tree/main/map-api/doc>

に各ステップ毎のコード状態を公開しているので併せて、
ご確認ください！

コピーする場合はGitHubからコピーしましょう

Today's Task

- やること
 - Mapbox の地図 API を使って地図を表示
- 使う言語
 - HTML / CSS / JavaScript (先週と一緒に)
- 事前準備
 - Mapbox のユーザ登録



The Three Components of a Web Application



HTML

部品と構造

CSS

デザイン

JavaScript

動き

All Steps

1. ディレクトリとファイルを用意する
2. Mapbox のJavaScript APIを読み込む
3. 地図を表示する領域を作成する
4. 表示させる地図の設定をする
5. 設定した領域に地図を表示する

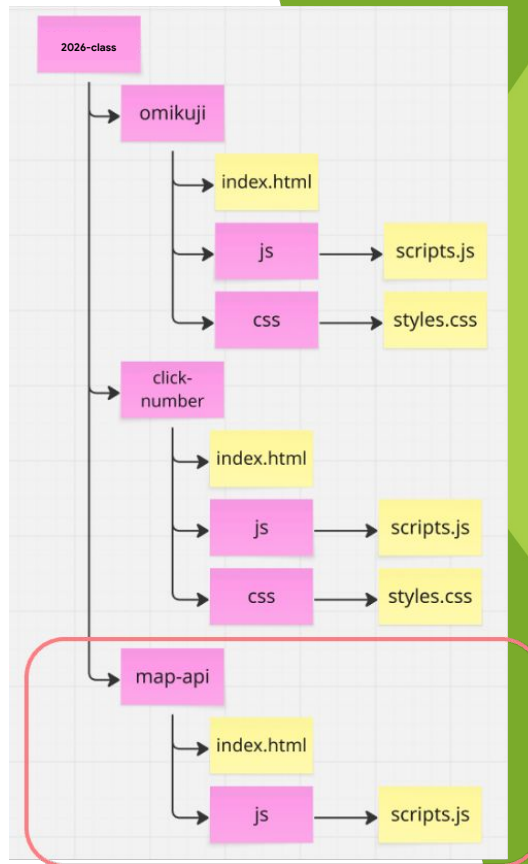
Task 1: Create the directory and files

先週作った omikuji / click-number と
同じ階層にもう1つディレクトリを作成しましょう。

右図のような構成でディレクトリと
ファイルを作成してください。

ファイルを作成したら step00.md から
各ファイルの中身をコピーしてください

- ピンク色: ディレクトリ
- 黄色: ファイル



All Steps

1. ディレクトリとファイルを用意する
2. **Mapbox のJavaScript APIを読み込む**
3. 地図を表示する領域を作成する
4. 表示させる地図の設定をする
5. 設定した領域に地図を表示する

Task 2: Load the Mapbox JavaScript API

Before
(index.html)

```
<title>地図を表示する - API Tutorial</title>
<meta name="viewport" content="initial-scale=1,maximum-scale=1,user-scalable=no" />
<!-- ここに Mapbox の JavascriptAPI と css を読み込むコードを書きます -->
```

After

```
<script src="https://api.mapbox.com/mapbox-gl-js/v3.24.0/mapbox-gl.js"></script>
<link href="https://api.mapbox.com/mapbox-gl-js/v3.24.0/mapbox-gl.css" rel="stylesheet" />
```

Task 2: Load the Mapbox JavaScript API

Before (index.html)

```
<title>地図を表示する - API Tutorial</title>
<meta name="viewport" content="initial-scale=1,maximum-scale=1,user-scalable=no" />
<!-- ここに Mapbox の JavascriptAPI と css を読み込むコードを書きます -->
```

After

```
<meta name="viewport" content="initial-scale=1,maximum-scale=1,user-scalable=no" />
<!-- ここに Mapbox の JavascriptAPI と css を読み込むコードを書きます -->
<script src="https://api.mapbox.com/mapbox-gl-js/v3.24.0/mapbox-gl.js"></script>
<link href="https://api.mapbox.com/mapbox-gl-js/v3.24.0/mapbox-gl.css" rel="stylesheet" />
```

```
<script src="https://api.mapbox.com/mapbox-gl-js/v3.24.0/mapbox-gl.js"></script>
<link href="https://api.mapbox.com/mapbox-gl-js/v3.24.0/mapbox-gl.css" rel="stylesheet" />
```

CSS ファイルを読みに行く時は **head タグ内** で呼び出しましょう。

JavaScript はどこから呼び出しても良いですが、head タグ内で呼び出すと JavaScript の処理が終わらないとページが読み込まれないので、**body タグ内で呼び出すのが良い** とされています。

※ 今回は head タグ内に書いてあります

All Steps

1. ディレクトリとファイルを用意する
2. Mapbox のJavaScript APIを読み込む
3. **地図を表示する領域を作成する**
4. 表示させる地図の設定をする
5. 設定した領域に地図を表示する

Task 3: Create a container for the map

Before
(index.html)

```
<body>  
<!-- ここにhtmlとjavascript書いていきます -->  
  
</body>
```

After

```
<div id="map"></div>
```


Task 3: Create a container for the map

Before
(index.html)

```
<body>
<!-- ここにhtmlとjavascript書いていきます -->

</body>
```

After

```
<body>
<!-- ここにhtmlとjavascript書いていきます -->
<div id="map"></div>

</body>
```

```
<div id="map"></div>
```

div タグは幅や余白を柔軟に変更することが可能なブロック要素。

まず、地図の表示領域を body 内に確保します。

ID は後ほど表示に使うために必ず記載するのを忘れずに。

All Steps

1. ディレクトリとファイルを用意する
2. Mapbox のJavaScript APIを読み込む
3. 地図を表示する領域を作成する
4. **表示させる地図の設定をする**
5. 設定した領域に地図を表示する

Prepare AccessToken for Mapbox

<https://console.mapbox.com/account/access-tokens/>

Access tokens

You need an API access token to configure [Mapbox GL JS](#), [Mobile](#), and [Mapbox web services](#) like routing and geocoding. Read more about [API access tokens](#) in our documentation.

+ Create a token

Name	Token	Last modified	URLs
Default public token Use protected tokens for live applications. See our token management best practices page to learn more.	pk. Bpt	about 2 years ago	N/A Refresh
map2025	pk. pot	about 1 year ago	0 ⋮

1. 画面遷移時に支払い方法を求められる可能性があるが Skip
2. Create a token(青枠) から Access Token を発行
3. 発行された Access Token はこのあと使うので画面をそのままにしておく

Task 4: Configure the map

Before
(index.html)

```
<body>
<!-- ここにhtmlとjavascript書いていきます -->
<div id="map"></div>

</body>
```

After

div タグの下に追加

```
<script>
// mapbox の accessToken を設定
mapboxgl.accessToken = '取得したaccessTokenを書く';
// 紀尾井町周辺の地図を表示させる
var map = new mapboxgl.Map({
  container: 'map',
  style: 'mapbox://styles/mapbox/outdoors-v11',
  center: [139.736897, 35.679584],
  zoom: 15
});
</script>
```

Task 4: Configure the map

Before
(index.html)

```
<body>
<!-- ここにhtmlとjavascript書いていきます -->
<div id="map"></div>

</body>
```

After

```
<body>
<!-- ここにhtmlとjavascript書いていきます -->
<div id="map"></div>
<script>
// mapbox の accessToken を設定
mapboxgl.accessToken = '取得したaccessTokenを書いてね';
// 紀尾井町周辺の地図を表示させる
var map = new mapboxgl.Map({
  container: 'map',
  style: 'mapbox://styles/mapbox/outdoors-v11',
  center: [139.736897, 35.679584],
  zoom: 15
});
</script>
</body>
```

div タグの下に追加

```
<script>
// mapbox の accessToken を設定
mapboxgl.accessToken = '取得したaccessTokenを書く';
// 紀尾井町周辺の地図を表示させる
var map = new mapboxgl.Map({
  container: 'map',
  style: 'mapbox://styles/mapbox/outdoors-v11',
  center: [139.736897, 35.679584],
  zoom: 15
});
</script>
```

JavaScript で地図の定義をしていきます。
まずは取得した accessToken を設定します。
ここは Mapbox を使う上で必要な定型文なので
コピーしちゃってください。
accessToken を設定しないと地図を表示すること
ができないので必ず書きましょう。

【参考】[Mapbox JavaScript API Reference](https://docs.mapbox.com/mapbox-gl-js/api/)

Task 4: Configure the map

Before
(index.html)

```
<body>
<!-- ここにhtmlとjavascript書いていきます -->
<div id="map"></div>

</body>
```

After

```
<body>
<!-- ここにhtmlとjavascript書いていきます -->
<div id="map"></div>
<script>
// mapbox の accessToken を設定
mapboxgl.accessToken = '取得したaccessTokenを書いてね';
// 紀尾井町周辺の地図を表示させる
var map = new mapboxgl.Map({
  container: 'map',
  style: 'mapbox://styles/mapbox/outdoors-v11',
  center: [139.736897, 35.679584],
  zoom: 15
});
</script>
</body>
```

div タグの下に追加

```
<script>
// mapbox の accessToken を設定
mapboxgl.accessToken = '取得したaccessTokenを書く';
// 紀尾井町周辺の地図を表示させる
var map = new mapboxgl.Map({
  container: 'map',
  style: 'mapbox://styles/mapbox/outdoors-v11',
  center: [139.736897, 35.679584],
  zoom: 15
});
</script>
```

次に地図の設定をします。地図オブジェクトのパラメータが何を示すかは下記を参照。

- container: Html の描画先を指定します
- style: 描画形式(衛星画像、白黒など)
- center: 地図の中心座標(経度、緯度)
- zoom: 数字が大きいほど拡大される

【参考】[Mapbox JavaScript API Reference](https://docs.mapbox.com/mapbox-gl-js/api/)

All Steps

1. ディレクトリとファイルを用意する
2. Mapbox のJavaScript APIを読み込む
3. 地図を表示する領域を作成する
4. 表示させる地図の設定をする
5. **設定した領域に地図を表示する**

Task 5: Display the map in the specified container

Before
(index.html)

```
<style>
/* ここにCSSを書きます */
</style>
```

After

```
body { margin: 0; padding: 0;}
#map { position: absolute; top: 0; bottom: 0; width: 100%; }
```


Task 5: Display the map in the specified container

Before
(index.html)

```
<style>
/* ここにCSSを書きます */
</style>
```

After

```
<style>
/* ここにCSSを書きます */
body { margin: 0; padding: 0;}
#map { position: absolute; top: 0; bottom: 0; width: 100%; }
</style>
```

```
body { margin: 0; padding: 0;}
#map { position: absolute; top: 0; bottom: 0; width: 100%; }
```

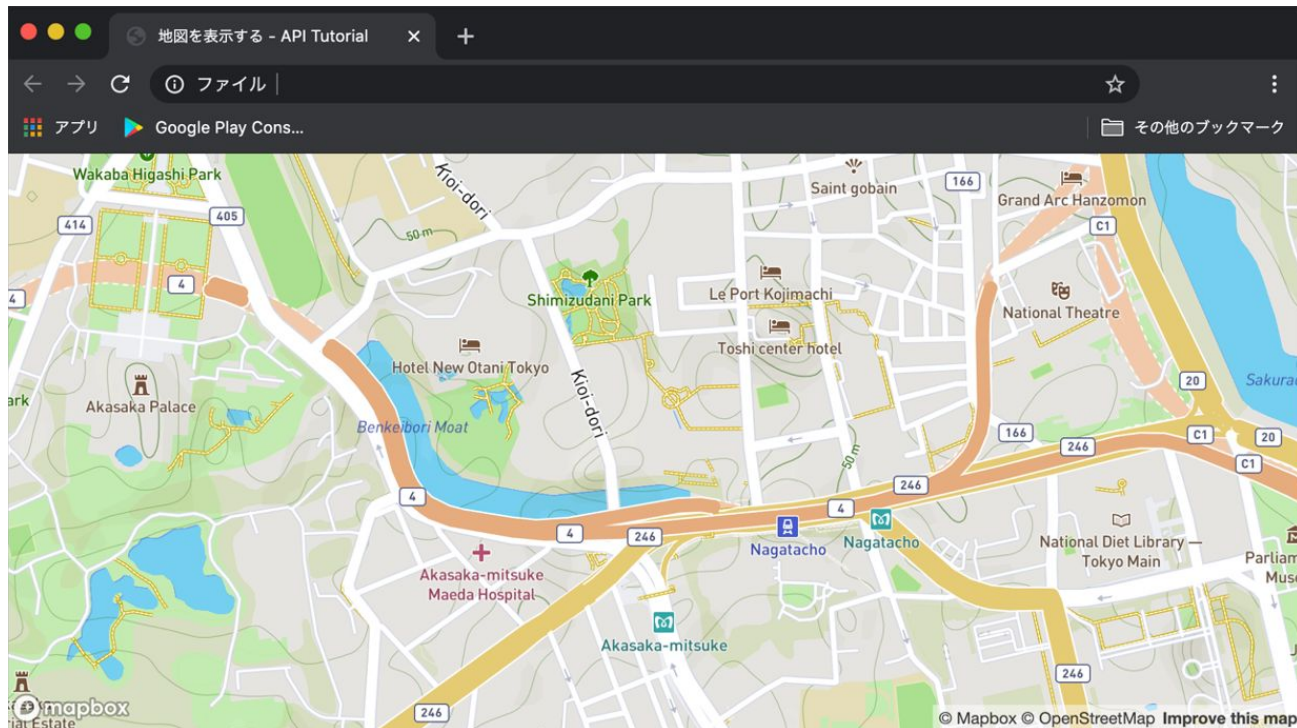
css で地図の表示領域を確保します(今回は全画面表示させます)。

div タグに設定した id に対して設置する位置と幅を設定することで地図が表示されます。

※ 高さは指定していませんが**初期値が auto で指定**されています。

Task 5: Display the map in the specified container

この画面になればOK！



【Advanced】Configure another map settings

余力のある人は地図の設定を変えてみましょう

- a. 地図の初期表示位置を変更する
- b. 地図の初期表示時のズームレベルを変える
- c. 地図のスタイルの種類を変更する

ドキュメントを見ながら他の変更をしてもOKです

ここまでやってみよう

Let's try this task

作業完了後、ファイルの保存を忘れずに
Don't forget to save the file

Explanation

【Recap】In-depth explanation

HTML と CSS の紐付け

CSS では右上のように、`.circle { ... }`
対して HTML の左上は、`class="circle"`

このように CSS で `.xxx { ... }`, HTML で `class="xxx"` と記述することによって、
CSS の `{ ... }` で定義したデザインを HTML 側に 反映させることができます。

HTML 上で `id="xxx"` Style 上で `#xxx { ... }`
HTML 上で `<xxx>` Style 上で `xxx { ... }`
といった形でもデザインを反映させることができます。

CSS

```
5  .circle {  
6      background-color: skyblue;  
7      border-radius: 30px;  
8      height: 60px;  
9      width: 60px;  
10     font-size: 24px;  
11     position: absolute;  
12 }
```

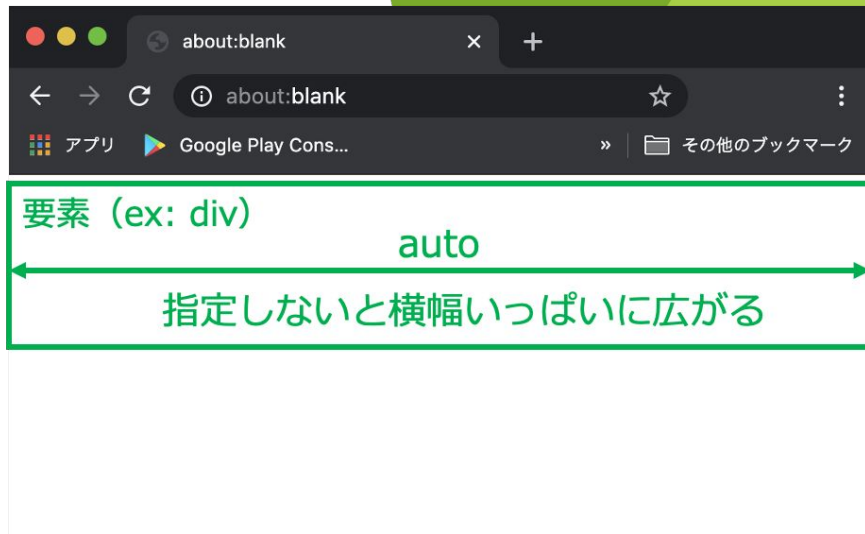
HTML

```
8  <body>  
9      <div id="app"></div>  
10     <div>  
11         <button class="circle">1</button>  
12     </div>  
13  
14     <script src="src/index.js"></script>
```

In-depth explanation

CSS でよく指定する width と height ですが、初期値はいずれも **auto** で指定されています。

auto の場合、幅は要素をフルで使いますが、高さはコンテンツに応じた高さになります。



In-depth explanation

CSS でよく指定する width と height ですが、初期値はいずれも **auto** で指定されています。

auto の場合、幅は要素をフルで使いますが、高さはコンテンツに応じた高さになります。

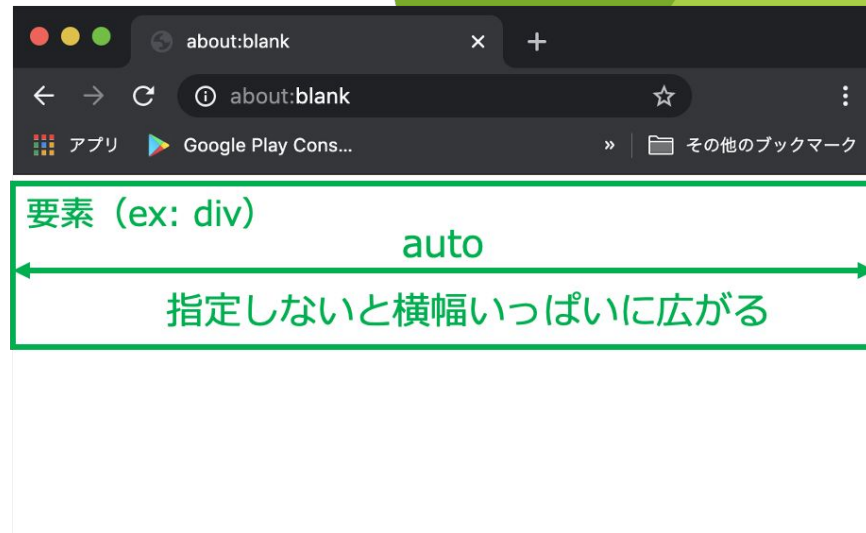
それでは以下の場合どうなるでしょうか？

HTML

```
<div id="parent">
  <p>ここに文章をたくさん書きますよ</p>
</div>
```

CSS

```
#parent { width: 100px; }
p { background: #00ff00; }
```



In-depth explanation

CSS でよく指定する width と height ですが、初期値はいずれも **auto** で指定されています。

auto の場合、幅は要素をフルで使いますが、高さはコンテンツに応じた高さになります。

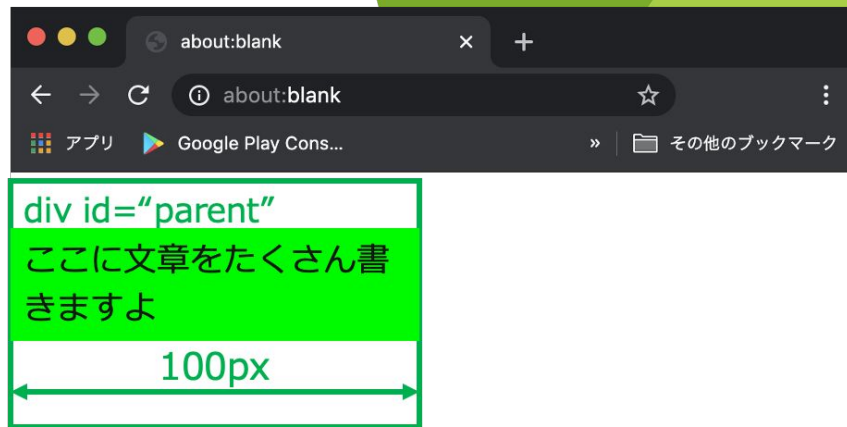
それでは以下の場合どうなるでしょうか？

HTML

```
<div id="parent">  
  <p>ここに文章をたくさん書きますよ</p>  
</div>
```

CSS

```
#parent { width: 100px; }  
p { background: #00ff00; }
```



親要素の幅が 100px に指定されているので、それ以上に長い文字が来ると勝手に折り返します。
※ 今回わかりやすく書いていますが、厳密に書くと div の高さは p タグの後ろところまでです。

In-depth explanation

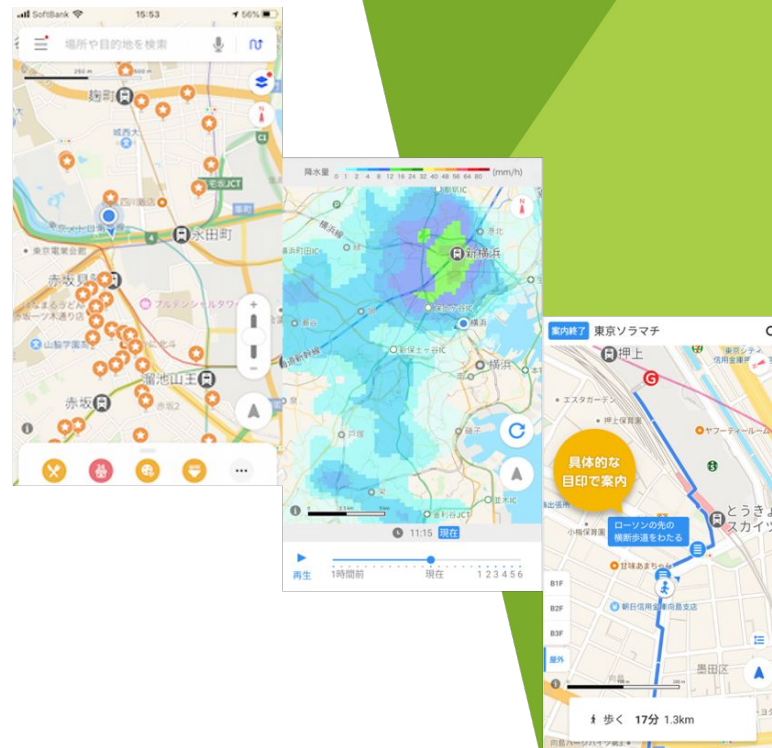
- API の使い方
 - JavaScript を読み込んで使う(今回はこっち)
 - Http を使ってリクエストして使う(こっちが一般的)
→ あとでお話します
- 地図を表示させる
 - **mapboxgl.accessToken**
 - Mapbox を使うために必要な定義
 - 誰が何回APIを呼び出しているのかなど管理している
→ リクエスト制限が設けられており、確認に利用
 - **Map object** ([Mapbox JavaScript API Reference](#))
 - 地図を最初に表示する内容を定義するオブジェクト
 - スタイルや中心座標はここで決める([スタイルの種類はここで確認してみよう](#))
 - ズームの限界を決めたりすることなど色々設定できます

Q : How is it different from existing map services?

Q : How is it different from existing map services?

- 検索ができない
- お気に入りが表示されない
- ルート案内ができない
- 雨雲レーダーでない
- 現在地と連動できない

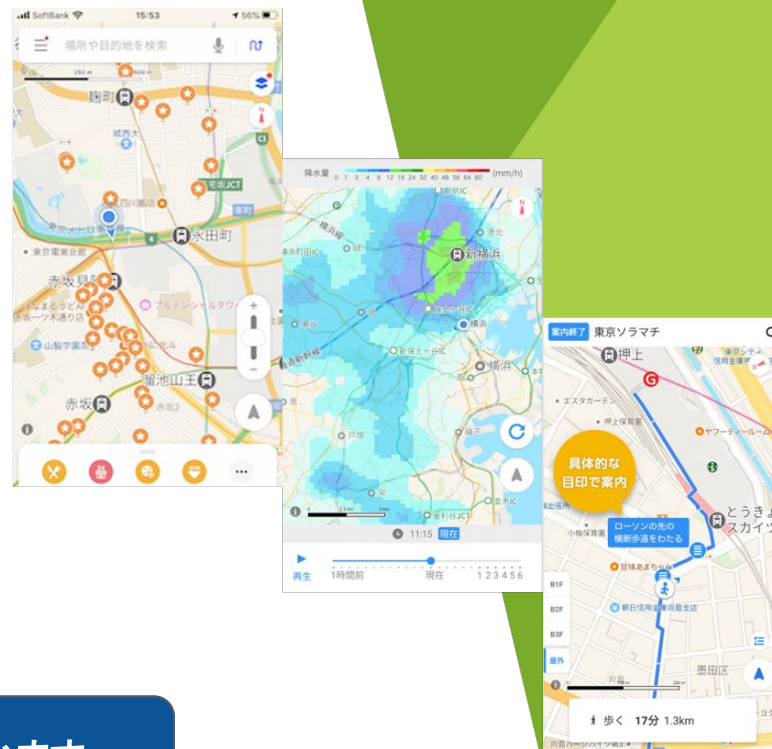
...etc



Q : How is it different from existing map services?

- 検索ができない
- お気に入りが表示されない
- ルート案内ができない
- 雨雲レーダーでない
- 現在地と連動できない

...etc



これらの機能はすべて APIとの組み合わせで実現されています

Q : How is it different from existing map services?

- 検索ができない → ローカル検索API
- お気に入りが表示されない → お気に入りAPI
- ルート案内ができない → ナビゲーションAPI
- 雨雲レーダーでない → 雨雲API
- 現在地と連動できない → 地図JavaScriptAPI に内包

An example of a product development

API をかけ合わせて作品概要を考える

→ さまざまな会社が Web API などデータを広く公開するようになったことで、誰でもかんたんにサービスを考えられるように。

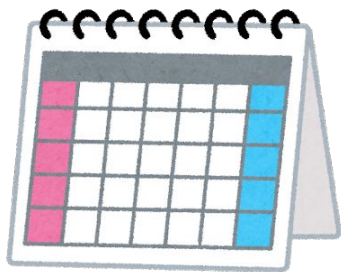
このように Web 上に公開されている情報を加工したり、編集することで新しいサービスを生み出すを
マッシュアップ といいます。

たとえば、、、

An example of a product development

カレンダーAPI x 天気予報API

予定が入っているカレンダーに天気を表示することで、そのまま予定を実行するのか、延期するのかを即時に判断できるようになる



x



Exercise

Let's MashUp

地図が表示できたので、地図と何かをマッシュアップさせてみましょう！！

- アイデアは自由
 - 地図上にこんなものが表示されると嬉しいなというものを考えてみてください
 - 2個だけでなく、3個4個マッシュアップしてもOKです
- 制限時間は5分

Example & Exercise

- 「地図」と「検索」をマッシュアップ
- 使う言語
 - HTML / CSS / JavaScript
- 使うAPI
 - Mapbox JavaScript API(継続)
 - Google Place API

※ Google Place API を使うには token の発行が必要です

この実習は任意とします。
実施する場合は、直前まで実施していた
API のディレクトリをコピーしてから実施してください

All Steps

1. ボタンを設置する
2. ボタンを表示させる
3. APIにリクエストする準備をする Part.1
4. APIにリクエストする準備をする Part.2
5. APIにリクエストする Part.1
6. APIにリクエストする Part.2
7. 仕上げ

Task 1: Create a new button

Before
(index.html)

```
<body>
<!-- ここにhtmlとjavascript書いていきます -->
<div id="map"></div>

</body>
```

After

div タグの上に追加

```
<div id="search-widget">
  <input type="text" id="search-term" placeholder="Enter a location">
  <button id="search-button">Search Hotels</button>
</div>
```

Task 1: Create a new button

Before
(index.html)

```
<body>
<!-- ここにhtmlとjavascript書いていきます -->
<div id="map"></div>

</body>
```

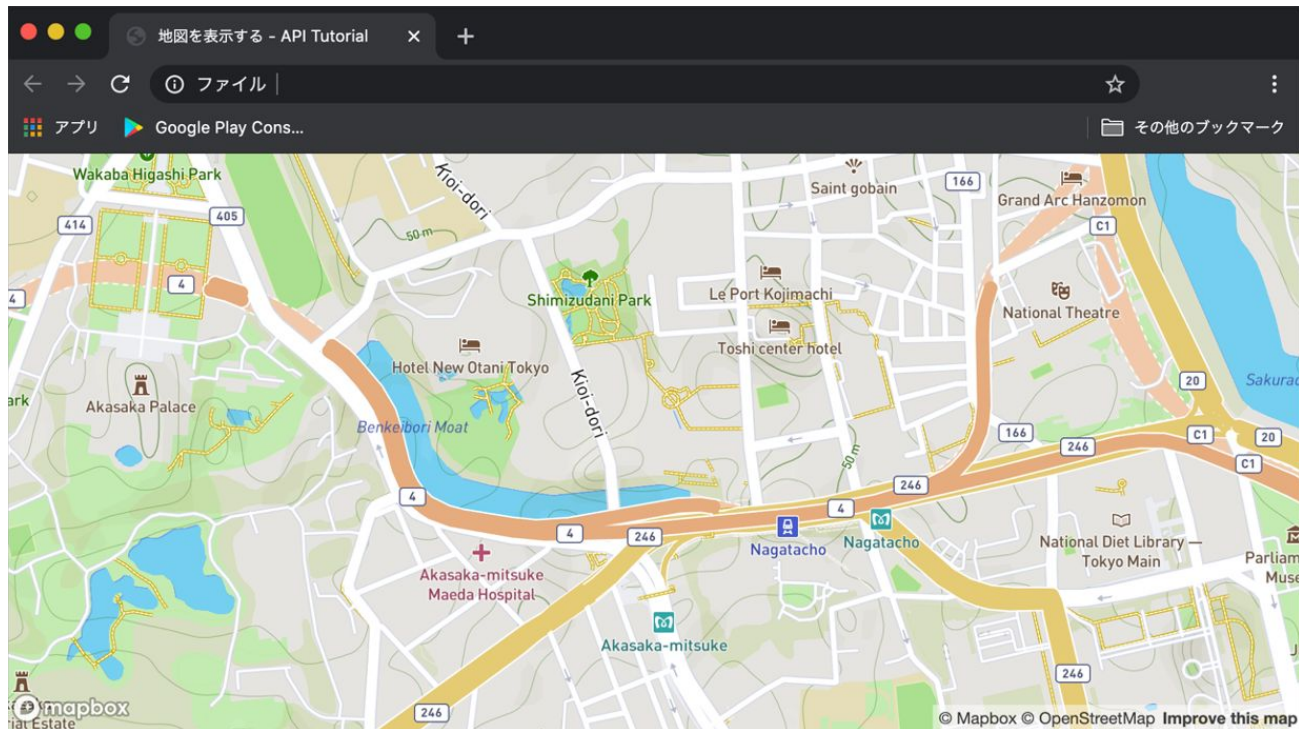
After

```
<div id="search-widget">
  <input type="text" id="search-term" placeholder="Enter a location">
  <button id="search-button">Search Hotels</button>
</div>
<div id="map"></div>
```

input タグは form タグと併用されることが多いですが、今回はテキスト(検索クエリ)を入力する枠を用意するために使うなので、input タグ単独で使います。
id はあとで css を当てるために設定、type はテキスト入力なので text を使い、placeholder には未入力時に表示されるテキストを入れておきます。
button タグは先週の授業でもやっているので割愛します。

Task 1: Create a new button

この画面になればOK！（ボタンは表示されないのが正解）



All Steps

1. ボタンを設置する
2. ボタンを表示させる
3. APIにリクエストする準備をする Part.1
4. APIにリクエストする準備をする Part.2
5. APIにリクエストする Part.1
6. APIにリクエストする Part.2
7. 仕上げ

Task 2: Display a new button

Before
(index.html)

```
<!-- ここに Mapbox の JavascriptAPI と css を読み込むコードを書きます -->
<script src="https://api.mapbox.com/mapbox-gl-js/v3.12.0/mapbox-gl.js"></script>
<link href="https://api.mapbox.com/mapbox-gl-js/v3.12.0/mapbox-gl.css" rel="stylesheet" />
<style>
/* ここにCSSを書きます */
body { margin: 0; padding: 0;}
#map { position: absolute; top: 0; bottom: 0; width: 100%; }
</style>
```

After

style タグの部分を削除して以下を追加

```
<link rel="stylesheet" href="css/styles.css">
```

Task 2: Display a new button

Before (index.html)

```
<!-- ここに Mapbox の JavascriptAPI と css を読み込むコードを書きます -->
<script src="https://api.mapbox.com/mapbox-gl-js/v3.12.0/mapbox-gl.js"></script>
<link href="https://api.mapbox.com/mapbox-gl-js/v3.12.0/mapbox-gl.css" rel="stylesheet" />
<style>
/* ここにCSSを書きます */
body { margin: 0; padding: 0;}
#map { position: absolute; top: 0; bottom: 0; width: 100%; }
</style>
```

After

```
<!-- ここに Mapbox の JavascriptAPI と css を読み込むコードを書きます -->
<script src="https://api.mapbox.com/mapbox-gl-js/v3.12.0/mapbox-gl.js"></script>
<link href="https://api.mapbox.com/mapbox-gl-js/v3.12.0/mapbox-gl.css" rel="stylesheet" />
<link rel="stylesheet" href="css/styles.css">
```

style タグの部分を削除して以下を追加

```
<link rel="stylesheet" href="css/styles.css">
```

Task 2: Display a new button

今回スタイルシートにいろいろ書いたので、別ファイル化します。

既存のものをすべて消して、スタイルシートを読み込む形にしてみました。

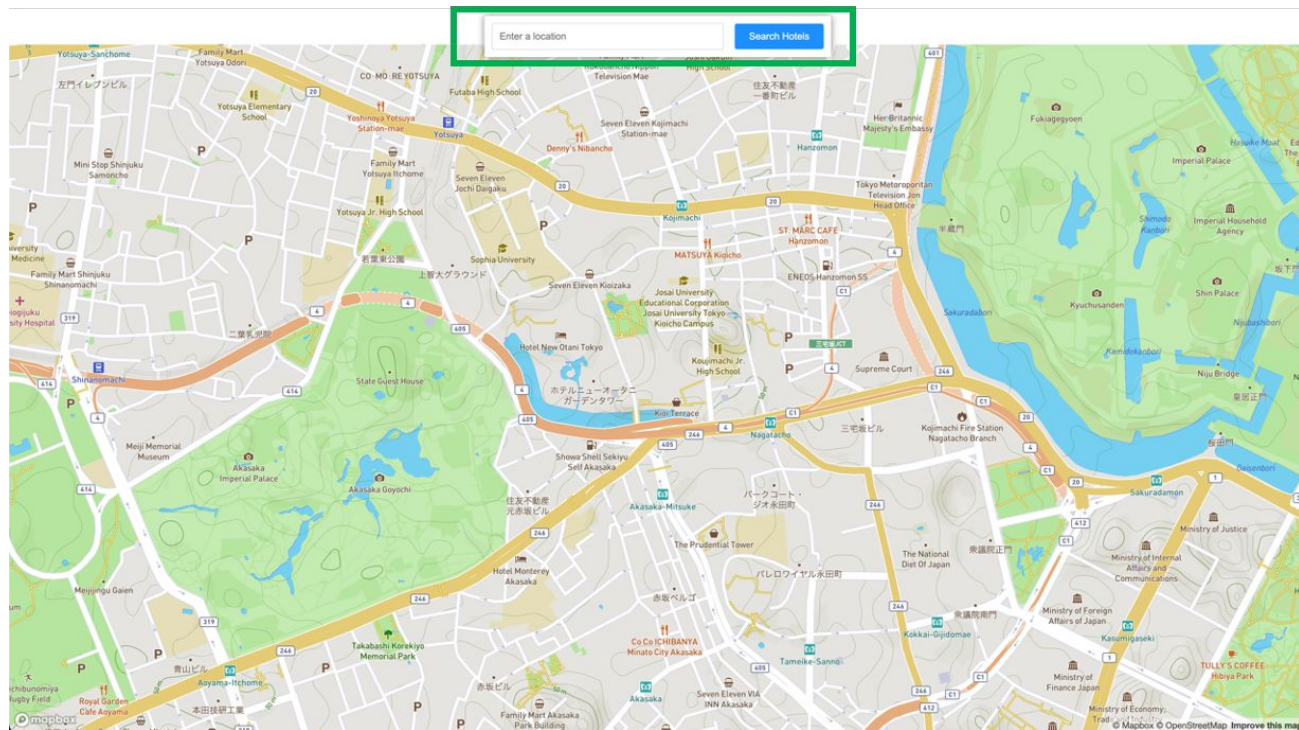
ディレクトリとファイルを作成したら以下をコピーしてください

<https://github.com/sonatax/class-2025/blob/main/map-api/doc/api02.md>

- 今回のポイント : **z-index**
 - 重なりの順番を定義(プラスマイナスの整数で指定)
 - 使うには position が absolute / relative / fixed が指定されている必要がある
 - 他の要素が指定されていなければ指定されているものが上にくる

Task 2: Display a new button

この画面になればOK！



All Steps

1. ボタンを設置する
2. ボタンを表示させる
3. **APIにリクエストする準備をする Part.1**
4. APIにリクエストする準備をする Part.2
5. APIにリクエストする Part.1
6. APIにリクエストする Part.2
7. 仕上げ

Task 3: Prepare for API request Part.1

Before
(index.html)

```
<script>
// mapbox の accessToken を設定
mapboxgl.accessToken = '取得したaccessTokenを書いてね';
// 紀尾井町周辺の地図を表示させる
var map = new mapboxgl.Map({
  container: 'map',
  style: 'mapbox://styles/mapbox/outdoors-v11',
  center: [139.736897, 35.679584],
  zoom: 15
});
</script>
```

After

script タグの部分を削除して以下を追加

```
<script src="js/scripts.js"></script>
```

Task 3: Prepare for API request Part.1

Before (index.html)

```
<script>
// mapbox の accessToken を設定
mapboxgl.accessToken = '取得したaccessTokenを書いてね';
// 紀尾井町周辺の地図を表示させる
var map = new mapboxgl.Map({
  container: 'map',
  style: 'mapbox://styles/mapbox/outdoors-v11',
  center: [139.736897, 35.679584],
  zoom: 15
});
</script>
```

After

```
<div id="search-widget">
  <input type="text" id="search-term" placeholder="Enter a location">
  <button id="search-button">Search Hotels</button>
</div>
<div id="map"></div>
<script src="js/scripts.js"></script>
```

script タグの部分を削除して以下を追加

```
<script src="js/scripts.js"></script>
```

Task 3: Prepare for API request Part.1

CSS同様に JavaScript も別ファイルで用意するので、読み込む形式に変更しています。

ディレクトリとファイルを作成したら以下をコピーしてください

<https://github.com/sonatax/class-2025/blob/main/map-api/doc/api03.md>

※ さっきまで書いてあったコードをファイルに移しただけです

All Steps

1. ボタンを設置する
2. ボタンを表示させる
3. APIにリクエストする準備をする Part.1
4. **APIにリクエストする準備をする Part.2**
5. APIにリクエストする Part.1
6. APIにリクエストする Part.2
7. 仕上げ

Task 4: Prepare for API request Part.2

Before (index.html)

```
<div id="search-widget">
  <input type="text" id="search-term" placeholder="Enter a location">
  <button id="search-button">Search Hotels</button>
</div>
<div id="map"></div>
<script src="js/scripts.js"></script>
```

After

```
<div id="map"></div>
<script src="script.js"></script>
<script src="https://maps.googleapis.com/maps/api/js?key=Google API KEY&libraries=places&callback=initMap" async defer></script>
```

script タグの下に以下を追加

```
<script src="https://maps.googleapis.com/maps/api/js?key=Google API
KEY&libraries=places&callback=initMap" async defer></script>
```

※ “Google API KEY” は自分で取得した API KEY に変更しないと動きません

Task 4: Prepare for API request Part.2

- js/scripts.js の一番下に追加
 - コードは GitHub を確認
 - <https://github.com/sonatax/class-2025/blob/main/map-api/doc/api04.md>
- ポイント
 - HTML の callback で指定した関数の呼び出し
 - 今回は Google Place API を使うので、地図を表示させずに Places サービスだけを作成
 - 検索ボタンのクリックを拾えるように準備

```
// map の初期化
function initMap() {
  // Googleマップインスタンスを作成
  // ここではマップを表示せず、Places Libraryの機能のみを使用します
  const map = new google.maps.Map(document.createElement('div'));
  // Placesサービスを作成
  const placesService = new google.maps.places.PlacesService(map);

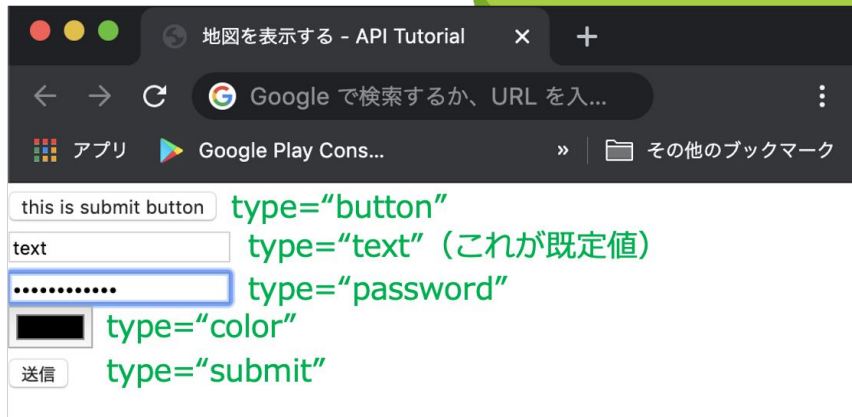
  // Mapboxでの検索ボタンクリックイベント
  document.getElementById('search-button').addEventListener('click',
function() {
    console.log("this is a debug code");
  });
}
```

Explanation

In-depth explanation : Task 3

input タグはさまざまな type を持っており、よくアンケートなどのフォームで利用されます。

その一例をいくつか書いておきましたが、パスワードなど利用に応じたものを選びながらフォームを形成していきます(ラジオボタンなども使えます)。



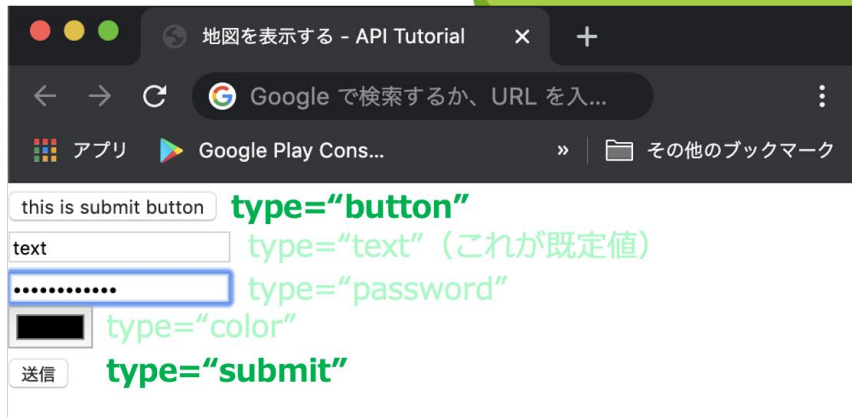
In-depth explanation : Task 3

input タグはさまざまな type を持っており、よくアンケートなどのフォームで利用されます。

その一例をいくつか書いておきましたが、パスワードなど利用に応じたものを選びながらフォームを形成していきます(ラジオボタンなども使えます)。

形成が終わったあとはボタンを押すと送信されますが、type の button と submit には差分があります。form で指定された際に submit を選ぶとデータが送信されますが、button の場合は、クリックされた際のイベントを取り、そこでデータを送信するプログラムを書く必要があります。

【参考】[MDN web docs - <input>: 入力欄\(フォーム入力\)要素](#)



In-depth explanation : Task 4

インスタンスとは？

- class をもとに作成されたオブジェクト
- 今回利用している class (設計図)
 - google.maps.Map
 - google.maps.places.PlacesService
- 作成されたインスタンス (設計に基づいたオブジェクト)
 - `const map = new google.maps.Map();`
 - `const placesService =`
`new google.maps.places.PlacesService(map);`
- インスタンスを作成する時は **new** を使って宣言する

In-depth explanation : Task 4

JavaScript が本当に動いているのかよくわからない時はデバッグコードを呼び出し箇所に入れることで確認することができます。

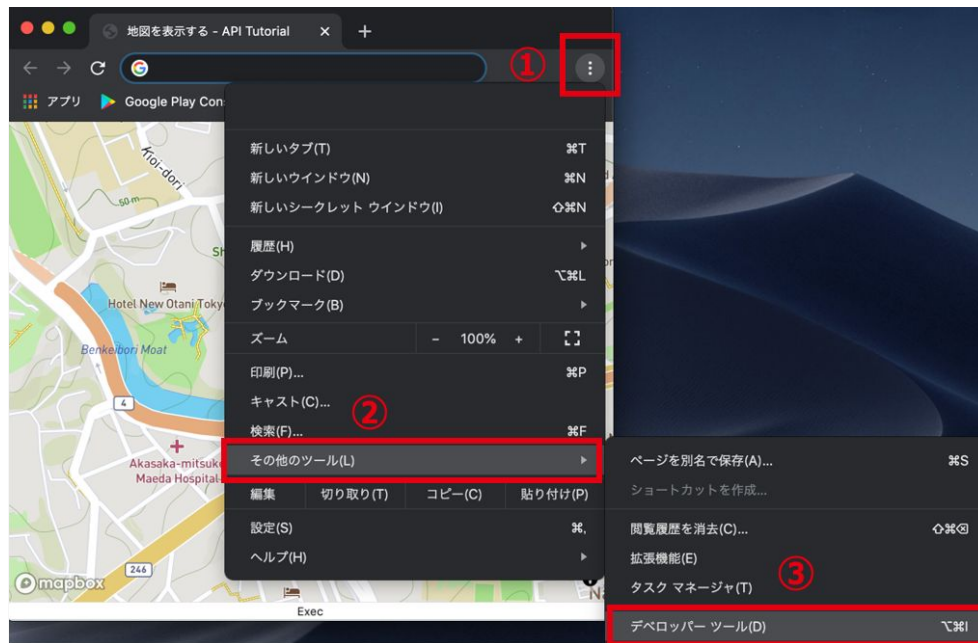
これでボタンが押されているかどうかの確認をすることができます。

今回は以下のコードが該当: `console.log("this is a debug code");`

```
// Mapboxでの検索ボタンクリックイベント
document.getElementById('search-button').addEventListener('click', function() {
    console.log("this is a debug code");
});
```


In-depth explanation : Task 4

※ 今回はGoogle Chrome を使ったデバッグのやり方
Chromeを開いたら画面右上にある三点リーダーから
「その他のツール」→「デベロッパーツール」をクリック

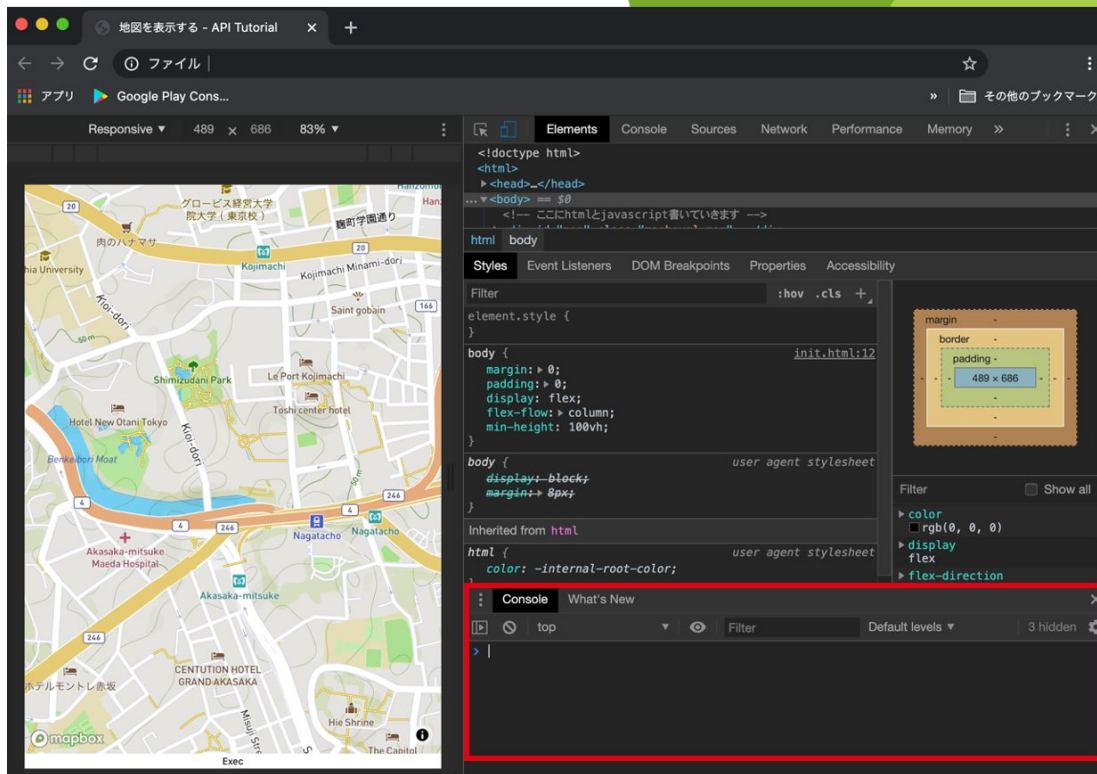


In-depth explanation : Task 4

こんな感じの画面が出てきたらOK

HTML や CSS などいろいろな要素が書かれています、画面下部の赤枠に注目
(Consoleのログ画面)

ここに console.log で仕込んだデバッグログが出力されます。



All Steps

1. ボタンを設置する
2. ボタンを表示させる
3. APIにリクエストする準備をする Part.1
4. APIにリクエストする準備をする Part.2
5. **APIにリクエストする Part.1**
6. APIにリクエストする Part.2
7. 仕上げ

Task 5: Send a request to APIs Part.1

Before
(scripts.js)

```
// Mapboxでの検索ボタンクリックイベント
document.getElementById('search-button').addEventListener('click', function() {
  console.log("this is a debug code");
});
```

After

console.log の行を消して、以下の md からソースをコピーする

<https://github.com/sonatax/class-2025/blob/main/map-api/doc/api05.md>

<https://github.com/sonatax/class-2025/tree/main/map-api/doc>

Task 5: Send a request to APIs Part.1

Before (scripts.js)

```
// Mapboxでの検索ボタンクリックイベント
document.getElementById('search-button').addEventListener('click', function() {
  console.log("this is a debug code");
});
```

After

```
// Mapboxでの検索ボタンクリックイベント
document.getElementById('search-button').addEventListener('click', function() {
  let searchTerm = document.getElementById('search-term').value;
  if (searchTerm) {
    fetch('https://api.mapbox.com/geocoding/v5/mapbox.places/' + encodeURIComponent(searchTerm) + '.json?access_token=' +
      process.env.MAPBOX_ACCESS_TOKEN)
      .then(response => response.json())
      .then(data => {
        if (data.features.length > 0) {
          const location = data.features[0].center;
          searchHotels(location[1], location[0], placesService);
          mapbox.flyTo({
            center: location,
            zoom: 14
          });
        } else {
          alert('Location not found');
        }
      })
      .catch(error => {
        console.error('Error:', error);
        alert('Failed to retrieve location data');
      });
  } else {
    alert('Please enter a location');
  }
});
```

console.log の行を消して、以下の md からソースをコピーする

<https://github.com/sonatax/class-2025/blob/main/map-api/doc/api05.md>

<https://github.com/sonatax/class-2025/tree/main/map-api/doc>

Explanation

In-depth explanation : Task 5

ポイント

- searchTerm には入力された場所情報
- fetch を使ってAPI にリクエスト
 - JavaScript に内包された関数で、
非同期なリクエスト をする時に
利用する
- 冒頭の if 文は入力されているかの確認

```
// Mapboxでの検索ボタンクリックイベント
document.getElementById('search-button').addEventListener('click', function() {
  let searchTerm = document.getElementById('search-term').value;
  if (searchTerm) {
    fetch('https://api.mapbox.com/geocoding/v5/mapbox.places/' + encodeURIComponent(searchTerm) + '.json?access_token=' + process.env.MAPBOX_ACCESS_TOKEN)
      .then(response => response.json())
      .then(data => {
        if (data.features.length > 0) {
          const location = data.features[0].center;
          searchHotels(location[1], location[0], placesService);
          mapbox.flyTo({
            center: location,
            zoom: 14
          });
        } else {
          alert('Location not found');
        }
      }).catch(error => {
        console.error('Error:', error);
        alert('Failed to retrieve location data');
      });
    } else {
      alert('Please enter a location');
    }
  });
});
```

※ 非同期処理とは

- JavaScript は基本的にシングルスレッド(単一の処理しかできない)
- APIにリクエストをしている間、レスポンスがないと動けなくなってしまう
- 非同期処理にすることで、JavaScript に書いてある他のコードが実行できるようにしている。

In-depth explanation : Task 5

ポイント

.then / .catch でレスポンスを取得

- .then の部分で API リクエストした結果を受け取る
 - a. 文字列を緯度経度に変換して結果を受け取る
 - b. 受け取った結果から必要なデータ（緯度経度）を取得し、周辺のホテルを探す関数を呼び出す（searchHotels は Task 6 で説明）
 - c. flyToで地図の中心座標を移動
- .catch はエラーが返されるので、エラー時の処理を記載していく

```
// Mapboxでの検索ボタンクリックイベント
document.getElementById('search-button').addEventListener('click', function() {
  let searchTerm = document.getElementById('search-term').value;
  if (searchTerm) {
    fetch('https://api.mapbox.com/geocoding/v5/mapbox.places/' + encodeURIComponent(searchTerm) + '.json?access_token=' + MAPBOX_ACCESS_TOKEN)
      .then(response => response.json())
      .then(data => {
        if (data.features.length > 0) {
          const location = data.features[0].center;
          searchHotels(location[1], location[0], placesService);
          mapbox.flyTo({
            center: location,
            zoom: 14
          });
        } else {
          alert('Location not found');
        }
      }).catch(error => {
        console.error('Error:', error);
        alert('Failed to retrieve location data');
      });
    } else {
      alert('Please enter a location');
    }
  });
});
```


All Steps

1. ボタンを設置する
2. ボタンを表示させる
3. APIにリクエストする準備をする Part.1
4. APIにリクエストする準備をする Part.2
5. APIにリクエストする Part.1
- 6. APIにリクエストする Part.2**
7. 仕上げ

Task 6: Prepare for API request Part.2

Before
(scripts.js)

```
mapboxgl.accessToken = '取得したaccessTokenを書いてね';  
  
// 地図を初期化する
```

After

accessToken の1行下に変数を追加
var allMarkers = [];

Task 6: Prepare for API request Part.2

Before
(scripts.js)

```
mapboxgl.accessToken = '取得したaccessTokenを書いてね';  
  
// 地図を初期化する
```

After

```
mapboxgl.accessToken = '取得したaccessTokenを書いてね';  
var allMarkers = [];  
  
// 地図を初期化する
```

accessToken の1行下に変数を追加

```
var allMarkers = [];
```

マーカー(ピン)を管理するための配列を用意
今回の配列はテーブル型の箱だと考えてください。

index	data
0	マーカー1の情報
1	マーカー2の情報
...	...

Task 6: Send a request to APIs Part.2

Before
(scripts.js)

ファイルの最下部に追加

After

initMap の下に新しく関数を追加していきます(js ファイルの最下部に以下の md の該当部分のみをコピーする)

<https://github.com/sonatax/class-2025/blob/main/map-api/doc/api06.md>

<https://github.com/sonatax/class-2025/tree/main/map-api/doc>

Task 6: Send a request to APIs Part.2

Before
(scripts.js)

ファイルの最下部に追加

After

```
// 位置 (緯度、経度) を基に周辺のホテル情報を検索する関数
function searchHotels(lat, lng, placesService) {
  // Places APIのリクエストを設定
  const request = {
    location: new google.maps.LatLng(lat, lng),
    radius: '3000', // 検索半径 (メートル)
    type: ['lodging'] // 宿泊施設を検索
  };

  // Google Places APIの周辺検索を実行
  placesService.nearbySearch(request, (results, status) => {
    if (status === google.maps.places.PlacesServiceStatus.OK && results) {

      // 検索結果を地図にマーカーとして表示
      results.forEach(place => {
        var popup = new mapboxgl.Popup({ offset: 25 })
          .setText(place.name);
        var marker = new mapboxgl.Marker()
          .setLngLat([place.geometry.location.lng(), place.geometry.location.lat()])
          .setPopup(popup)
          .addTo(mapbox);

        allMarkers.push(marker);
      });
    } else {
      alert('No hotels found');
    }
  });
}
```

initMap の下に新しく関数を追加していきます(js ファイルの最下部に以下の md の該当部分のみをコピーする)

<https://github.com/sonatax/class-2025/blob/main/map-api/doc/api06.md>

Explanation

In-depth explanation : Task 6

ポイント

引数に緯度経度とインスタンスを設定

- 関数内部でAPI をリクエストするため、引数に緯度経度を設定してデータを渡している
- placesService は initMap した時のものをそのまま流用する
 - 同じインスタンスを複数作ると処理がどんどん重くなるので注意

```
// 位置（緯度、経度）を基に周辺のホテル情報を検索する関数
function searchHotels(lat, lng, placesService) {
  // Places APIのリクエストを設定
  const request = {
    location: new google.maps.LatLng(lat, lng),
    radius: '3000', // 検索半径（メートル）
    type: ['lodging'] // 宿泊施設を検索
  };

  // Google Places APIの周辺検索を実行
  placesService.nearbySearch(request, (results, status) => {
    if (status === google.maps.places.PlacesServiceStatus.OK && results) {

      // 検索結果を地図にマーカーとして表示
      results.forEach(place => {
        var popup = new mapboxgl.Popup({ offset: 25 })
          .setText(place.name);
        var marker = new mapboxgl.Marker()
          .setLngLat([place.geometry.location.lng(), place.geometry.location.lat()])
          .setPopup(popup)
          .addTo(mapbox);

        allMarkers.push(marker);
      });
    } else {
      alert('No hotels found');
    }
  });
}
```

※ 設定の詳細は [Google Place API のドキュメント](#)を確認

In-depth explanation : Task 6

ポイント

Google Place API にリクエストする設定

- 検索する場所(location)は先程引数に設定した緯度経度を使う
- radius で緯度経度を中心とした検索の範囲を指定する
- type で検索の結果を絞り込む(今回は宿泊施設を選択)

```
// 位置 (緯度、経度) を基に周辺のホテル情報を検索する関数
function searchHotels(lat, lng, placesService) {
  // Places APIのリクエストを設定
  const request = {
    location: new google.maps.LatLng(lat, lng),
    radius: '3000', // 検索半径 (メートル)
    type: ['lodging'] // 宿泊施設を検索
  };

  // Google Places APIの周辺検索を実行
  placesService.nearbySearch(request, (results, status) => {
    if (status === google.maps.places.PlacesServiceStatus.OK && results) {

      // 検索結果を地図にマーカーとして表示
      results.forEach(place => {
        var popup = new mapboxgl.Popup({ offset: 25 })
          .setText(place.name);
        var marker = new mapboxgl.Marker()
          .setLngLat([place.geometry.location.lng(), place.geometry.location.lat()])
          .setPopup(popup)
          .addTo(mapbox);

        allMarkers.push(marker);
      });
    } else {
      alert('No hotels found');
    }
  });
}
```

※ 設定の詳細は [Google Place API のドキュメント](#)を確認

In-depth explanation : Task 6

ポイント

nearbySearch メソッドを使ってリクエスト

- メソッド自体にリクエストする機構が内包
- リクエストした結果は results, status に返却されるので、それぞれを確認してから定常処理とエラー処理を記載
- results のデータを使ってマーカーと吹き出しを表示させる
 - forEach を使っていますが、これまでに教えた for 文でも実行可能

```
// 位置（緯度、経度）を基に周辺のホテル情報を検索する関数
function searchHotels(lat, lng, placesService) {
  // Places APIのリクエストを設定
  const request = {
    location: new google.maps.LatLng(lat, lng),
    radius: '3000', // 検索半径（メートル）
    type: ['lodging'] // 宿泊施設を検索
  };

  // Google Places APIの周辺検索を実行
  placesService.nearbySearch(request, (results, status) => {
    if (status === google.maps.places.PlacesServiceStatus.OK && results) {

      // 検索結果を地図にマーカーとして表示
      results.forEach(place => {
        var popup = new mapboxgl.Popup({ offset: 25 })
          .setText(place.name);
        var marker = new mapboxgl.Marker()
          .setLngLat([place.geometry.location.lng(), place.geometry.location.lat()])
          .setPopup(popup)
          .addTo(mapbox);

        allMarkers.push(marker);
      });
    } else {
      alert('No hotels found');
    }
  });
}
```

※ 設定の詳細は [Google Place API のドキュメント](#)を確認

In-depth explanation : Task 6

今回は Mapbox が提供しているライブラリを使って

Popup / Marker を表示

- **Popup**

- offset の数字はポップアップの表示場所 → ピクセルで指定する

- **Marker**

- setLngLat は API で取得した結果からマーカーの座標を指定する
- setPopup で前述の popup を指定してマーカーとポップアップを紐づけ
- addTo(mapbox) で地図に追加
- 事前に用意しておいた allMarkers(配列)にマーカー情報を追加

```
// 位置（緯度、経度）を基に周辺のホテル情報を検索する関数
function searchHotels(lat, lng, placesService) {
  // Places APIのリクエストを設定
  const request = {
    location: new google.maps.LatLng(lat, lng),
    radius: '3000', // 検索半径（メートル）
    type: ['lodging'] // 宿泊施設を検索
  };

  // Google Places APIの周辺検索を実行
  placesService.nearbySearch(request, (results, status) => {
    if (status === google.maps.places.PlacesServiceStatus.OK && results) {

      // 検索結果を地図にマーカーとして表示
      results.forEach(place => {
        var popup = new mapboxgl.Popup({ offset: 25 })
          .setText(place.name);
        var marker = new mapboxgl.Marker()
          .setLngLat([place.geometry.location.lng(), place.geometry.location.lat()])
          .setPopup(popup)
          .addTo(mapbox);

        allMarkers.push(marker);
      });
    } else {
      alert('No hotels found');
    }
  });
}
```

All Steps

1. ボタンを設置する
2. ボタンを表示させる
3. APIにリクエストする準備をする Part.1
4. APIにリクエストする準備をする Part.2
5. APIにリクエストする Part.1
6. APIにリクエストする Part.2
7. **仕上げ**

Task 7 : Finishing Touches

- Task 6 完了時点で検索は利用可能
- 地図を使った検索サービスとしては未完成

Q. 何が足りないのでしょうか？

Task 7 : Finishing Touches

- Task 6 完了時点で検索は利用可能
- 地図を使った検索サービスとしては未完成

Q. 何が足りないのでしょうか？

A. 検索する度に結果が地図上に残り続けてしまう

Task 7 : Finishing Touches

Before (scripts.js)	ファイルの最下部に追加
After	

searchHotels の下に新しく関数を追加 (js ファイルの最下部に以下の md の該当部分のみをコピーする)

<https://github.com/sonatax/class-2025/blob/main/map-api/doc/api07.md>

Task 7 : Finishing Touches

Before
(scripts.js)

ファイルの最下部に追加

After

```
// delete all markers
function clearMarkers() {
  // 保存されているすべてのマーカーをループし削除
  for (var i = 0; i < allMarkers.length; i++) {
    allMarkers[i].remove(); // Mapbox GL JSのremove() メソッドを使用
  }
  // 配列をリセット
  allMarkers = [];
}
```

searchHotels の下に新しく関数を追加(js ファイルの最下部に以下の md の該当部分のみをコピーする)

<https://github.com/sonatax/class-2025/blob/main/map-api/doc/api07.md>

Task 7 : Finishing Touches

Before (scripts.js)

```
// Google Places APIの周辺検索を実行
placesService.nearbySearch(request, (results, status) => {
  if (status === google.maps.places.PlacesServiceStatus.OK && results) {

    // 検索結果を地図にマーカーとして表示
```

After

searchHotel 関数の検索が正常に動いたあとに以下を追加

```
clearMarkers();
```


Task 7 : Finishing Touches

Before (scripts.js)

```
// Google Places APIの周辺検索を実行
placesService.nearbySearch(request, (results, status) => {
  if (status === google.maps.places.PlacesServiceStatus.OK && results) {

    // 検索結果を地図にマーカーとして表示
```

After

```
// Google Places APIの周辺検索を実行
placesService.nearbySearch(request, (results, status) => {
  if (status === google.maps.places.PlacesServiceStatus.OK && results) {
    // 現在の地図上のマーカーをクリアする
    clearMarkers();

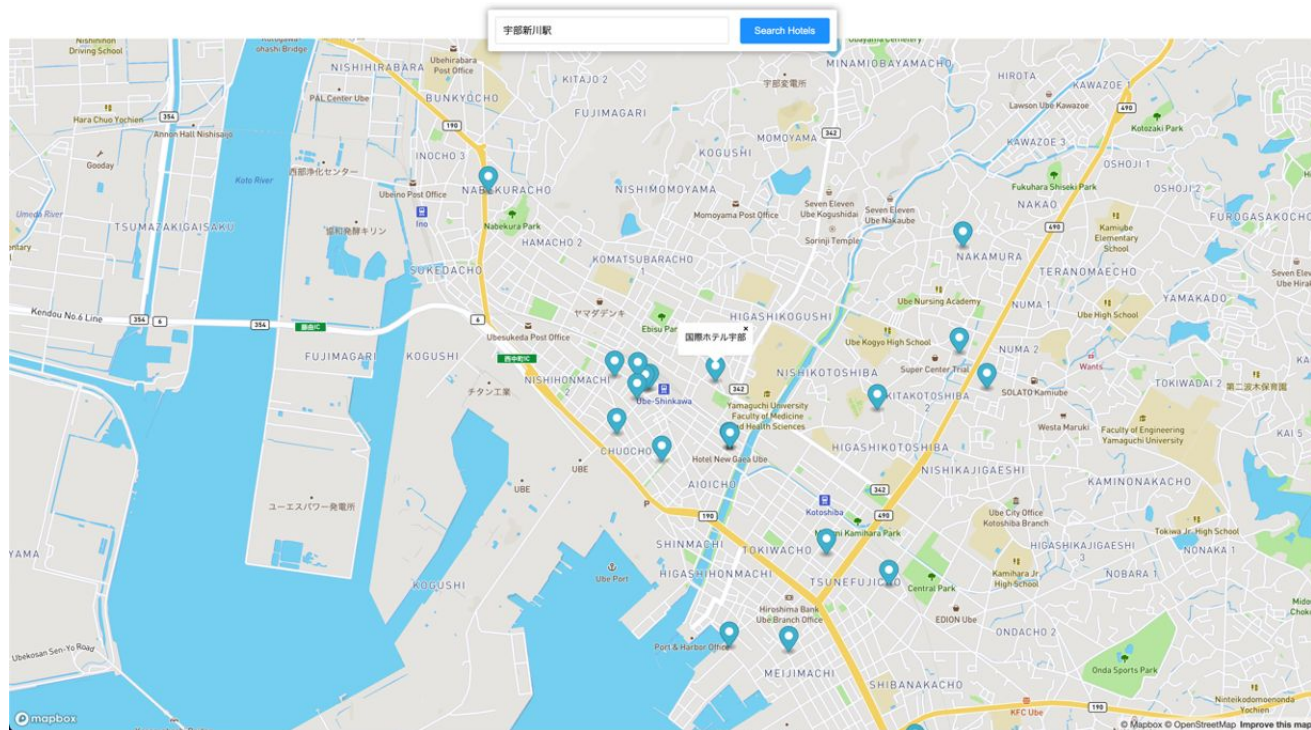
    // 検索結果を地図にマーカーとして表示
```

searchHotel 関数の検索が正常に動いたあとに以下を追加

```
clearMarkers();
```

Task 7 : Finishing Touches

この画面になればOK！



Completed!!!

【Advanced】Read data from the file

- オープンデータのファイルを読み込んでマーカーとして表示させてみましょう
 - 今回は宇部市のトイレ情報を表示させてみる
- ソースコード
 - <https://github.com/sonatax/class-2025/blob/main/map-api/doc/extension01.md>
- All Steps
 - Task 1 : ボタンの追加 (HTML / CSS の編集)
 - Task 2 : ボタン押下時の処理の追加 (JavaScript の編集)

Task 1 : Add a data button

Before
(index.html)

```
<div id="map"></div>  
<script src="script.js"></script>
```

After

div タグと script タグの間に追加

```
<button id="bottom-right-button">Display Data</button>
```

Task 1 : Add a data button

Before
(index.html)

```
<div id="map"></div>
<script src="script.js"></script>
```

After

```
<div id="map"></div>
<!-- ここで新しくボタンを追加 -->
<button id="bottom-right-button">Display Data</button>
<script src="js/scripts.js"></script>
```

div タグと script タグの間に追加

```
<button id="bottom-right-button">Display Data</button>
```

Task 1 : Add a data button

Before
(styles.css)

ファイルの最下部に追加

After

styles.css の最下部に以下を追加

```
#bottom-right-button:hover {  
    background-color: #007bff;  
}
```

Task 1 : Add a data button

Before
(styles.css)

ファイルの最下部に追加

After

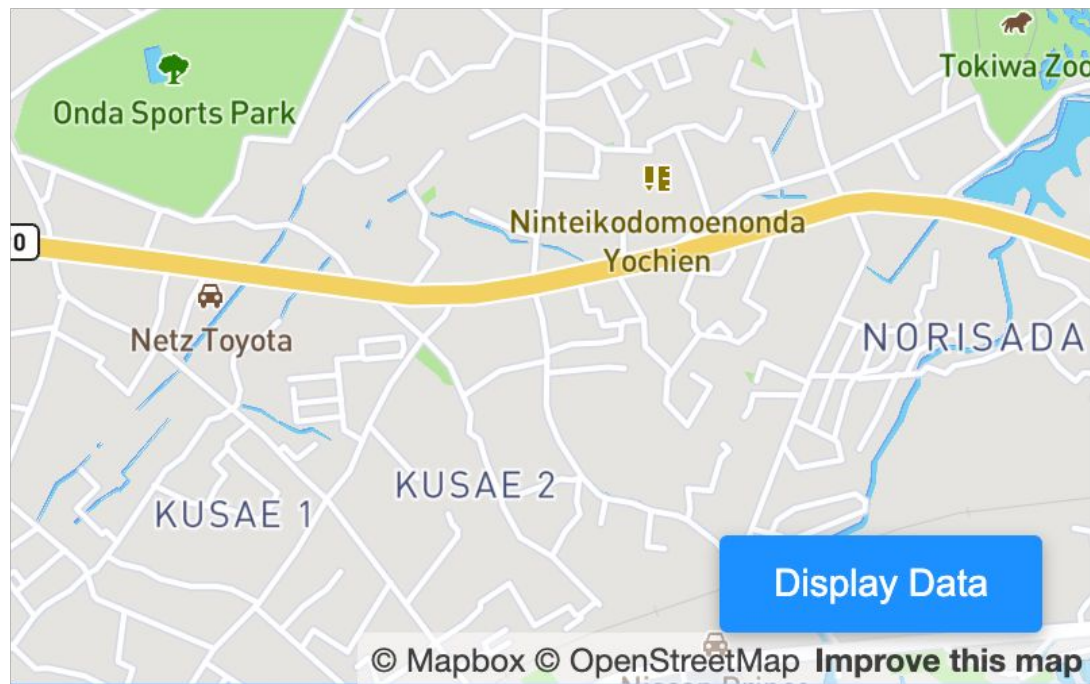
```
#bottom-right-button:hover {  
    background-color: #007bff;  
}
```

styles.css の最下部に以下を追加

```
#bottom-right-button:hover {  
    background-color: #007bff;  
}
```


Task 1 : Add a data button

こんな感じで画面右下にボタンが出ればOK !



Task 2 : Adding behavior to button clicks

Before
(scripts.js)

ファイルの最下部に追加

After

```
// 右下ボタンがクリックされたときにCSVを読み込んでマーカーを立てる
document.getElementById('bottom-right-button').addEventListener('click', function() {
  fetch('https://raw.githubusercontent.com/sonatax/class-2025/refs/heads/main/map-api/data/toilet.csv')
    .then(response => {
      if (!response.ok) {
        throw new Error('CSV の読み込みに失敗しました: ' + response.status);
      }
      return response.text();
    })
    .then(text => {
      const lines = text.trim().split('\n');
      lines.forEach(line => {
        // CSVの各行を「番号,公園名,緯度,経度」で想定
        const cols = line.split(',');
        if (cols.length >= 4) {
          const name = cols[1];
          const lat = parseFloat(cols[2]);
          const lng = parseFloat(cols[3]);
          if (!isNaN(lat) && !isNaN(lng)) {
            // ポップアップ付きマーカーを作成
            const popup = new mapboxgl.Popup({ offset: 25 }).setText(name);
            const marker = new mapboxgl.Marker({ color: '#FF0000' })
              .setLngLat([lng, lat])
              .setPopup(popup)
              .addTo(mapbox);
            allMarkers.push(marker);
          }
        }
      });
    })
    .catch(error => {
      console.error('Error loading CSV:', error);
      alert('トイレデータの読み込みに失敗しました');
    });
});
```

scripts.js の最下部にボタン押下時の処理を追加(js ファイルの最下部に以下の md の該当部分のみをコピーする)

<https://github.com/sonatax/class-2025/blob/main/map-api/doc/extension01.md>

こんな感じで検索のピンと違う色のピンが出たらOK!



Explanation

In-depth explanation : Advanced Task 2

ポイント

fetch でファイルを読み込みにいく

.then / .catch でレスポンスを取得

- .then で response を取得してリクエストが成功したかどうかを判定
 - 失敗した場合はエラーメッセージを投げる
 - 成功したら取得したデータの text を return する

```
// 右下ボタンがクリックされたときにCSVを読み込んでマーカーを立てる
document.getElementById('bottom-right-button').addEventListener('click', function() {
  fetch('https://raw.githubusercontent.com/sonatax/class-2025/refs/heads/main/map-api/data/toilet.csv')
    .then(response => {
      if (!response.ok) {
        throw new Error('CSV の読み込みに失敗しました: ' + response.status);
      }
      return response.text();
    })
    .then(text => {
      const lines = text.trim().split('\n');
      lines.forEach(line => {
        // CSVの各行を「番号,公園名,緯度,経度」で想定
        const cols = line.split(',');
        if (cols.length >= 4) {
          const name = cols[1];
          const lat = parseFloat(cols[2]);
          const lng = parseFloat(cols[3]);
          if (!isNaN(lat) && !isNaN(lng)) {
            // ポップアップ付きマーカーを作成
            const popup = new mapboxgl.Popup({ offset: 25 }).setText(name);
            const marker = new mapboxgl.Marker({ color: 'FF0000' })
              .setLngLat([lng, lat])
              .setPopup(popup)
              .addTo(mapbox);
            allMarkers.push(marker);
          }
        }
      });
    })
    .catch(error => {
      console.error('Error loading CSV:', error);
      alert('トイレデータの読み込みに失敗しました');
    });
});
```

In-depth explanation : Advanced Task 2

ポイント

return されたデータを次の .then で処理

- lines に1行ごとのデータを配列として格納
- lines のデータを forEach を使って1行ごとに読み込んで処理をしていく

※ forEach は **配列の長さの分処理を繰り返し** て実行してくれるので for 文とうまく使い分けて利用しましょう

```
// 右下ボタンがクリックされたときにCSVを読み込んでマーカーを立てる
document.getElementById('bottom-right-button').addEventListener('click', function() {
  fetch('https://raw.githubusercontent.com/sonatax/class-2025/refs/heads/main/map-api/data/toilet.csv')
    .then(response => {
      if (!response.ok) {
        throw new Error('CSV の読み込みに失敗しました: ' + response.status);
      }
      return response.text();
    })
    .then(text => {
      const lines = text.trim().split('\n');
      lines.forEach(line => {
        // CSVの各行を「番号,公園名,緯度,経度」で想定
        const cols = line.split(',');
        if (cols.length >= 4) {
          const name = cols[1];
          const lat = parseFloat(cols[2]);
          const lng = parseFloat(cols[3]);
          if (!isNaN(lat) && !isNaN(lng)) {
            // ポップアップ付きマーカーを作成
            const popup = new mapboxgl.Popup({ offset: 25 }).setText(name);
            const marker = new mapboxgl.Marker({ color: 'FF0000' })
              .setLngLat([lng, lat])
              .setPopup(popup)
              .addTo(mapbox);
            allMarkers.push(marker);
          }
        }
      });
    })
    .catch(error => {
      console.error('Error loading CSV:', error);
      alert('トイレデータの読み込みに失敗しました');
    });
});
```

In-depth explanation : Advanced Task 2

ポイント

- 1行ごとのデータを split で分割
- 分割したデータの長さが4以上ない場合はマーカのデータが足りないなので何も処理をしない
- データがある場合は lat / lng でマーカの座標を指定して、吹き出しにテキストを設定
 - この辺はさっきと同じ処理

```
// 右下ボタンがクリックされたときにCSVを読み込んでマーカを立てる
document.getElementById('bottom-right-button').addEventListener('click', function() {
  fetch('https://raw.githubusercontent.com/sonatax/class-2025/refs/heads/main/map-api/data/toilet.csv')
    .then(response => {
      if (!response.ok) {
        throw new Error('CSV の読み込みに失敗しました: ' + response.status);
      }
      return response.text();
    })
    .then(text => {
      const lines = text.trim().split('\n');
      lines.forEach(line => {
        // CSVの各行を「番号,公園名,緯度,経度」で想定
        const cols = line.split(',');
        if (cols.length >= 4) {
          const name = cols[1];
          const lat = parseFloat(cols[2]);
          const lng = parseFloat(cols[3]);
          if (!isNaN(lat) && !isNaN(lng)) {
            // ポップアップ付きマーカを作成
            const popup = new mapboxgl.Popup({ offset: 25 }).setText(name);
            const marker = new mapboxgl.Marker({ color: 'FF0000' })
              .setLngLat([lng, lat])
              .setPopup(popup)
              .addTo(mapbox);
            allMarkers.push(marker);
          }
        }
      });
    })
    .catch(error => {
      console.error('Error loading CSV:', error);
      alert('トイレデータの読み込みに失敗しました');
    });
});
```

Next Week's Agenda

- 前半: 中間発表
 - 1チーム3分で現状報告
 - 発表資料は**必須**(授業後に提出)
 - 発表後には各チームの現状を踏まえて講師陣からフィードバック
- 後半: 各チームごとの作業時間